

AI for Clinical Study Reports and Submission

Yilong Zhang

Nan Xiao

Table of contents

Welcome	4
Preface	5
In this book	5
Authors and contributors	5
 I Principles	 6
1 Principles overview	7
1.1 A simple model	7
1.2 Principle 1: Define the workflow boundary	8
1.3 Principle 2: Treat deliverables as typed, versioned artifacts	8
1.4 Principle 3: Design events and rules before agent behavior	9
1.5 Principle 4: Build narrow, deterministic tools	10
1.6 Principle 5: Let the harness control the workflow	10
1.7 Principle 6: Require evidence-backed outputs	11
1.8 Principle 7: Design human escalation as part of the workflow	12
1.9 Principle 8: Test the workflow at multiple layers	12
1.10 Principle 9: Deploy where the artifacts live and monitor the workflow	13
1.11 Principle 10: Design for iteration after changes	14
1.12 Applying the principles to the agent example	14
1.12.1 Workflow objective	14
1.12.2 Agent role	14
1.12.3 Inputs	15
1.12.4 Trigger examples	15
1.12.5 Tool set	15
1.12.6 Workflow stages	15
1.12.7 Example checks	16
1.12.8 Expected outputs	16
1.13 Summary	16

II	Documentation as code	17
2	Documentation overview	18
2.1	TBD	18
III	Analysis and reporting	19
3	Reporting overview	20
3.1	TBD	20
IV	Governance	21
4	Governance overview	22
4.1	TBD	22
	References	23

Welcome

Welcome to AI for Clinical Study Reports and Submission. Clinical study reports (CSRs) sit at the heart of clinical trial development. They bring the design, analysis, and interpretation of a study together in an integrated document.

This book shows how to use AI to plan, build, and automate CSRs, from data and analyses to tables, figures, and listings. We follow the [ICH E3 guidance on structure and content](#) and demonstrate practical, reproducible workflows that help you meet those expectations.

Whether you are new to AI-native workflows or already working in clinical reporting, you will find small, reusable examples you can adopt right away, along with patterns and checklists for organizing work for regulatory submission and effective team collaboration.

This is a work-in-progress draft.

Preface

In this book

This book is designed for people who are interested in using AI for clinical development. Each part of the book makes certain assumptions about the readers' background:

- Part 1, TBD
- Part 2, TBD
- Part 3, TBD
- Part 4, TBD

Authors and contributors

This document is a collaborative effort maintained by a community. As you read through it, you also have the opportunity to contribute and enhance its quality. Your input and involvement play a vital role in shaping the excellence of this document.

- Authors: made significant contributions to at least one chapter, constituting the majority of the content.

[Yilong Zhang](#), [Nan Xiao](#),

Part I

Principles

1 Principles overview

Objective

This chapter introduces practical principles for designing an AI-first workflow for clinical study work. We use one running example throughout: an AI agent acting as an independent reviewer assistant to verify that a CSR TLF package derived from ADaM datasets follows the SAP.

An AI-first workflow is not simply a human workflow with an LLM added at the end. It is designed so an AI agent can perform the first structured pass, use tools to inspect study artifacts, produce evidence-backed findings, and rerun when events and rules indicate that the work has changed.

The accountable human remains the reviewer and decision-maker. The agent helps by doing systematic, repetitive, evidence-gathering work.

1.1 A simple model

For this book, we use a practical model:

AI agent = LLM + tools + harness + running environment

- **LLM:** reasons over the task, interprets structured outputs, and drafts findings.
- **Tools:** inspect real study artifacts, such as SAP text, ADaM metadata, datasets, TLF outputs, logs, and specifications.
- **Harness:** controls the workflow loop, routes tool calls, stores state, applies rules, and decides when to stop or escalate.
- **Running environment:** provides the file system, compute environment, permissions, credentials, and execution context where the workflow runs.

The design goal is not to make the LLM guess better. The design goal is to give the agent a workflow, tools, rules, evidence requirements, and escalation paths so it can behave like a reliable independent reviewer assistant.

1.2 Principle 1: Define the workflow boundary

Start by defining the workflow using handoff points, not tools.

A useful boundary statement answers four questions:

1. What artifacts does the agent receive?
2. What question is the agent allowed to answer?
3. What output does the agent produce?
4. What remains a human responsibility?

For the running example:

Given a specific SAP version, ADaM metadata snapshot, ADaM dataset snapshot, and TLF package, the agent produces a structured consistency report and escalation list for human review.

In scope:

- identify SAP requirements relevant to selected TLFs;
- inspect ADaM variable metadata, dataset structure, and analysis flags;
- inspect TLF titles, footnotes, columns, populations, and displayed values;
- compare SAP expectations with ADaM and TLF evidence;
- produce findings with source references.

Out of scope:

- deciding whether the SAP is scientifically appropriate;
- making final statistical judgments;
- editing study deliverables directly;
- replacing formal organizational review, validation, or sign-off.

1.3 Principle 2: Treat deliverables as typed, versioned artifacts

The agent should not treat inputs as random files. Each artifact should have a type, version, owner, and role in the workflow.

For the running example:

Artifact	Role in workflow	Example version key
SAP	Source of planned analyses and display requirements	amendment number, date, document ID

Artifact	Role in workflow	Example version key
ADaM metadata	Source of dataset and variable structure	define.xml version, metadata freeze ID
ADaM datasets	Source of analysis data and population flags	data cut, transfer ID, checksum
TLF package	Source of generated CSR displays	output run ID, table/listing/figure ID
Agent report	Evidence output for review	agent run ID

Typed artifacts make reruns easier. If only ADaM metadata changes, the workflow can identify which checks depend on changed variables instead of rerunning everything blindly.

1.4 Principle 3: Design events and rules before agent behavior

An AI-first workflow is iterative. It should rerun when the work changes.

An **event** says something happened. A **rule** decides what the agent should do next.

Examples:

Event	Rule	Rerun scope
SAP amendment issued	Re-map SAP requirements	Full or affected endpoint review
ADaM metadata updated	Compare metadata differences	Checks touching changed datasets or variables
ADaM datasets refreshed	Recompute affected evidence	Population and value checks
TLF output regenerated	Compare display changes	Affected TLFs
Human reviewer requests recheck	Run targeted review	Requested endpoint, table, or finding
Another AI agent updates a deliverable	Verify downstream impact	Rule-dependent

This is a core part of AI-first design. The workflow does not wait for a human to remember every follow-up check. The harness watches for events, applies rules, and triggers the agent when useful.

1.5 Principle 4: Build narrow, deterministic tools

Tools should answer focused questions and return structured data. They should not return long prose that the LLM has to reinterpret.

Good tool design principles:

- **Narrow:** one tool answers one kind of question.
- **Read-only by default:** review workflows should inspect before they modify.
- **Structured:** return JSON, tables, or typed records.
- **Idempotent:** same input snapshot returns the same output.
- **Loud on failure:** missing files or unreadable metadata should create explicit errors.

For the running example, useful tools might include:

Tool	Purpose	Example output
<code>parse_sap_requirements()</code>	Extract endpoints, populations, estimands, displays, and required analyses from SAP sections	endpoint-to-display map
<code>inspect_adam_metadata()</code>	Inspect ADaM variable metadata	dataset, variable, label, type, codelist, analysis flag
<code>query_adam_population()</code>	Count subjects or records for a population flag	population count with dataset reference
<code>extract_tlf_metadata()</code>	Read titles, footnotes, columns, and N values from TLF outputs	display metadata
<code>compare_expected_actual()</code>	Compare expected SAP/ADaM/TLF elements	pass/fail finding record

The ADaM metadata tool is especially useful as an early example. It can inspect dataset names, variable names, labels, types, controlled terminology, and analysis flags so the agent can compare what exists in the data against what the SAP or TLF specification requires.

1.6 Principle 5: Let the harness control the workflow

The harness is the operational layer around the LLM. It should control the workflow structure so the agent does not become an open-ended conversation.

Harness responsibilities include:

- start the workflow from an event or human request;

- select the relevant artifact versions;
- route tool calls;
- store intermediate state;
- enforce stopping conditions;
- rerun only affected checks when possible;
- collect findings and escalations;
- produce final outputs.

For the running example, a simple harness can follow three stages:

1. **Map**: map SAP requirements to ADaM datasets, variables, population flags, and TLF IDs.
2. **Verify**: run checks for each mapped requirement.
3. **Synthesize**: group findings, summarize evidence, and prepare human review outputs.

This structure keeps the LLM focused. The LLM reasons inside a workflow; it does not invent the workflow from scratch every time.

1.7 Principle 6: Require evidence-backed outputs

The output of an AI-first workflow should be evidence, not just a summary.

Every finding should answer:

- What was checked?
- What was expected?
- What was observed?
- Where did the evidence come from?
- What is the severity?
- Who should review it?

A finding record could look like this:

```
{
  "finding_id": "F-012",
  "display_id": "T_14_2_1",
  "check_type": "population_mismatch",
  "expected": "Per-protocol population based on PPROTFL",
  "actual": "TLF title indicates Safety Population; ADaM metadata includes SAFFL and PPROTFL",
  "evidence": {
    "sap_section": "9.1.2",
    "adam_dataset": "ADSL",
    "adam_variables": ["SAFFL", "PPROTFL"],
```

```

    "tlf_file": "t_14_2_1.rtf"
  },
  "severity": "needs_review",
  "suggested_reviewer": "study statistician"
}

```

The human-readable report should be a rendering of structured evidence. That makes the workflow easier to review, test, compare across runs, and reuse.

1.8 Principle 7: Design human escalation as part of the workflow

Escalation is not failure. It is a designed output.

The agent should escalate when:

- SAP language is ambiguous;
- a required artifact is missing;
- a tool returns inconsistent evidence;
- a check requires statistical judgment;
- the agent cannot confidently map SAP text to ADaM or TLF elements.

Escalations should be routed by role:

Escalation type	Suggested reviewer
endpoint definition ambiguity	study statistician
missing or inconsistent ADaM metadata	statistical programmer
display formatting or table shell mismatch	TLF reviewer or programming lead
repeated blocking findings across many displays	programming lead

A good escalation is self-contained. The reviewer should see the source references, observed evidence, why the agent could not resolve it, and what decision is needed.

1.9 Principle 8: Test the workflow at multiple layers

Testing an AI-first workflow is not only testing whether tools run. We need to test whether the system produces correct findings and escalates appropriately.

Recommended test layers:

- **Tool tests:** each tool returns the expected structured output for known fixtures.
- **Workflow tests:** the harness runs the expected sequence for a given event.
- **Golden-set evaluation:** curated SAP/ADaM/TLF examples with known findings.
- **Adversarial cases:** ambiguous SAP language, missing variables, label mismatches, population mismatches, and harmless formatting changes.
- **Regression tests:** rerun prior examples after prompt, model, tool, or rule changes.

For the running example, a golden set might include:

Case	Expected behavior
all mappings correct	no active findings; complete evidence matrix
population flag missing	blocking finding and programmer escalation
SAP population wording ambiguous	statistician escalation
TLF title changed but data unchanged	display-level finding only
ADaM label changed	rerun checks touching that variable

1.10 Principle 9: Deploy where the artifacts live and monitor the workflow

The agent should run in an environment where it can access the right artifact versions safely and reproducibly.

For this demo, we focus on workflow design rather than full compliance implementation. In a real organization, deployment details depend on internal infrastructure, access controls, validation processes, and policies.

Even in a demo, the workflow should track operational signals:

- run status: completed, failed, or escalated;
- coverage: which endpoints, datasets, and TLFs were checked;
- finding distribution: blocking, needs review, informational, pass;
- escalation rate and routing accuracy;
- runtime and cost;
- changes between runs.

Monitoring should tell us whether the workflow is useful, not just whether it finished.

1.11 Principle 10: Design for iteration after changes

Clinical study deliverables change repeatedly. AI-first workflows should be designed for dry runs, updates, partial reruns, and delta reports.

Useful state across runs includes:

- artifact versions used in each run;
- rules applied;
- tools called;
- findings produced;
- human decisions or acknowledgments;
- findings resolved, repeated, or newly introduced.

For the running example, after an ADaM metadata update, the agent should not simply issue the same full report again. It should identify:

- findings fixed since the previous run;
- new findings introduced by the change;
- prior findings still unresolved;
- acknowledged deviations that should remain visible but not distract reviewers.

This turns the agent from a one-time reviewer into an iterative reviewer assistant that follows the study as it evolves.

1.12 Applying the principles to the agent example

The running example can be designed as the following AI-first workflow.

1.12.1 Workflow objective

Verify that selected CSR TLF outputs derived from ADaM datasets are consistent with SAP requirements before database lock.

1.12.2 Agent role

The agent acts as an independent reviewer assistant. It performs mechanical consistency checks, prepares evidence-backed findings, and escalates judgment calls to humans.

1.12.3 Inputs

- SAP version or amendment;
- ADaM metadata snapshot;
- ADaM dataset snapshot;
- TLF package;
- workflow rules defining when to run or rerun checks.

1.12.4 Trigger examples

- SAP section updated;
- ADaM metadata refreshed;
- ADaM datasets regenerated;
- TLF output regenerated;
- human requests targeted recheck;
- another AI agent changes a related artifact.

1.12.5 Tool set

- SAP requirement parser;
- ADaM metadata inspector;
- ADaM population and summary query tool;
- TLF metadata extractor;
- consistency checker;
- report generator.

1.12.6 Workflow stages

1. **Map requirements:** identify SAP endpoints, populations, required variables, and target TLFs.
2. **Inspect artifacts:** use tools to inspect ADaM metadata, ADaM data summaries, and TLF outputs.
3. **Check consistency:** compare SAP expectations against ADaM and TLF evidence.
4. **Classify findings:** assign severity, source references, and suggested reviewer role.
5. **Escalate judgment calls:** route ambiguous or high-impact findings to humans.
6. **Produce evidence report:** create structured output plus a readable summary.
7. **Track state:** compare with previous runs to identify fixed, new, repeated, and acknowledged findings.

1.12.7 Example checks

Check	Expected evidence
SAP population is represented in ADaM	population flag exists in ADSL or relevant dataset
SAP endpoint variable exists	variable exists with expected label/type in ADaM metadata
TLF title matches SAP display intent	display title/subtitle align with SAP or shell
TLF N values are explainable	counts can be reproduced from ADaM population rules
footnotes match analysis method	footnote text aligns with SAP method description
rerun scope is correct	event/rule mapping explains why checks ran

1.12.8 Expected outputs

- structured evidence matrix;
- human-readable review summary;
- finding list grouped by severity;
- escalation memo routed by role;
- delta report comparing current and previous runs.

1.13 Summary

Designing an AI-first workflow means designing the work so an AI agent can execute a structured, tool-supported, evidence-producing review cycle. The important design questions are not only “what can the LLM answer?” but also:

- What artifacts does the workflow depend on?
- What tools expose those artifacts reliably?
- What events and rules trigger the agent?
- What evidence must the agent produce?
- When does the agent escalate?
- How do we test and monitor the workflow?
- How does the workflow rerun as the study changes?

For clinical study workflows, this framing keeps the agent useful and bounded: it becomes an independent reviewer assistant that follows the evolution of study deliverables and helps humans focus on judgment, interpretation, and sign-off.

Part II

Documentation as code

2 Documentation overview

 Objective

TBD

2.1 TBD

TBD

Part III

Analysis and reporting

3 Reporting overview

 Objective

TBD

3.1 TBD

TBD

Part IV

Governance

4 Governance overview

 Objective
TBD

4.1 TBD

TBD

References